

EXPERIMENT 4

```
[1]
✓ 4s !pip install transformers torch --quiet

[2]
✓ 14s import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from IPython.display import Markdown, display

[3]
✓ 0s device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

Using device: cuda

[4]
✓ 16s ms_model_name = "microsoft/DialogPT-medium"

ms_tokenizer = AutoTokenizer.from_pretrained(ms_model_name)
ms_model = AutoModelForCausalLM.from_pretrained(ms_model_name).to(device)

def generate_microsoft(prompt):
    inputs = ms_tokenizer.encode(prompt + ms_tokenizer.eos_token, return_tensors='pt').to(device)

    outputs = ms_model.generate(
        inputs,
        max_length=80,
        pad_token_id=ms_tokenizer.eos_token_id,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )

    return ms_tokenizer.decode(outputs[:, inputs.shape[-1]:][0], skip_special_tokens=True)

... /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart the notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% ██████████ 642/642 [00:00<00:00, 65.4kB/s]
tokenizer_config.json: 100% ██████████ 614/614 [00:00<00:00, 60.8kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
vocab.json: 100% ██████████ 1.04M/? [00:00<00:00, 30.4MB/s]
merges.txt: 100% ██████████ 456k/? [00:00<00:00, 26.1MB/s]
pytorch_model.bin: 100% ██████████ 863M/863M [00:06<00:00, 76.8MB/s]
model.safetensors: 100% ██████████ 863M/863M [00:08<00:00, 210MB/s]
Loading weights: 100% ██████████ 293/293 [00:01<00:00, 280.57it/s, Materializing param=transformer.wte.weight]
The tied weights mapping and config for this model specifies to tie transformer.wte.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them.
GPT2LMHeadModel LOAD REPORT from: microsoft/DialogPT-medium
Key | Status |
-----+-----+
transformer.h.{0...23}.attn.bias | UNEXPECTED |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100% ██████████ 124/124 [00:00<00:00, 1.38kB/s]
```

EXPERIMENT 4

```
distil_model_name = "distilgpt2"

distil_tokenizer = AutoTokenizer.from_pretrained(distil_model_name)
distil_model = AutoModelForCausalLM.from_pretrained(distil_model_name).to(device)

def generate_distilgpt2(prompt):
    inputs = distil_tokenizer(prompt, return_tensors="pt").to(device)

    outputs = distil_model.generate(
        **inputs,
        max_length=80,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )

    return distil_tokenizer.decode(outputs[0], skip_special_tokens=True)
```

*** config.json: 100% ██████████ 762/762 [00:00<00:00, 13.5kB/s]
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 1.79kB/s]
vocab.json: 100% ██████████ 1.04M/1.04M [00:00<00:00, 4.10MB/s]
merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 2.56MB/s]
tokenizer.json: 100% ██████████ 1.36M/1.36M [00:00<00:00, 6.93MB/s]
model.safetensors: 100% ██████████ 353M/353M [00:02<00:00, 249MB/s]
Loading weights: 100% ██████████ 76/76 [00:00<00:00, 336.77it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2

```
gpt2_name = "gpt2"

gpt2_tokenizer = AutoTokenizer.from_pretrained(gpt2_name)
gpt2_model = AutoModelForCausalLM.from_pretrained(gpt2_name).to(device)

def generate_gpt2(prompt):
    inputs = gpt2_tokenizer(prompt, return_tensors="pt").to(device)

    outputs = gpt2_model.generate(
        **inputs,
        max_length=80,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )

    return gpt2_tokenizer.decode(outputs[0], skip_special_tokens=True)
```

*** config.json: 100% ██████████ 665/665 [00:00<00:00, 40.2kB/s]
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 1.08kB/s]
vocab.json: 100% ██████████ 1.04M/1.04M [00:00<00:00, 4.01MB/s]
merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 3.30MB/s]
tokenizer.json: 100% ██████████ 1.36M/1.36M [00:00<00:00, 6.02MB/s]
model.safetensors: 100% ██████████ 548M/548M [00:02<00:00, 266MB/s]
Loading weights: 100% ██████████ 148/148 [00:00<00:00, 539.91it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |

EXPERIMENT 4

```
prompt = "Write a haiku about AI and Nature"

competitors = []
answers = []

ms_response = generate_microsoft(prompt)
competitors.append("Microsoft DialogPT")
answers.append(ms_response)

distil_response = generate_distilgpt2(prompt)
competitors.append("DistilGPT2")
answers.append(distil_response)

gpt2_response = generate_gpt2(prompt)
competitors.append("GPT2")
answers.append(gpt2_response)

... Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.

for comp, ans in zip(competitors, answers):
    display(Markdown(f"## 🏆 {comp}\n\n{ans}\n\n---"))

for comp, ans in zip(competitors, answers):
    display(Markdown(f"## 🏆 {comp}\n\n{ans}\n\n---"))

... 🏆 Microsoft DialogPT
Neat. But the poem is bad.

🏆 DistilGPT2
Write a haiku about AI and Nature.

🏆 GPT2
Write a haiku about AI and Nature.
The main idea behind this haiku is to try to explain how the world works to others. The goal is to understand how the world works, but also to make an analogy to humans. The main idea behind this haiku is to try to explain how the world works, but also to make an analogy to humans. The main idea behind this haiku is to try to explain how the world works, but also to make an analogy to humans.

+ Code + Text

def score_response(text):
    words = text.split()
```

EXPERIMENT 4

```
def score_response(text):
    words = text.split()
    unique_words = len(set(words))
    total_words = len(words)

    repetition_penalty = total_words - unique_words
    length_penalty = abs(total_words - 17) # ideal haiku word count approx

    return repetition_penalty + length_penalty

scores = []

for ans in answers:
    scores.append(score_response(ans))

ranking = sorted(zip(scores, competitors), key=lambda x: x[0])

print("\n 🏆 FINAL RANKING (Best → Worst)\n")

for i, (_, model_name) in enumerate(ranking, 1):
    print(f"Rank {i}: {model_name}")

...

🏆 FINAL RANKING (Best → Worst)

Rank 1: Microsoft DialoGPT
Rank 2: DistilGPT2
Rank 3: GPT2
```